

Accord — Implementation Roadmap

Status

Draft v0.1

Purpose

This document converts the Accord thesis, PRD, protocol, architecture, and safety model into a practical implementation sequence.

It answers:

- what should be built first,
- what dependencies exist between components,
- how the work should be phased,
- what each milestone should prove,
- and how to keep the project from expanding faster than it becomes usable.

This roadmap is intentionally focused on **v1 execution discipline**.

1. Roadmap Principles

Before listing phases, the project needs a few operating rules.

1.1 Build the control plane before the demos

The protocol, validation, policy, and approval flow come first. Sample apps are proof, not the foundation.

1.2 Prefer vertical slices over isolated packages

Each milestone should prove an end-to-end behavior, not just produce folders and types.

1.3 Keep the first version local-first and inspectable

Avoid introducing hosted infrastructure, distributed sync, or broad third-party integrations early.

1.4 Treat documentation as part of implementation

The spec, ADRs, examples, and README are part of the product, not cleanup work.

1.5 Earn scope expansion

No new adapter, workflow, or policy sophistication should be added until the existing control flow is real and testable.

2. Overall Delivery Strategy

The strongest delivery pattern for Accord is:

1. **Define the runtime grammar clearly**
2. **Make it executable locally**
3. **Prove the fail-closed approval boundary**
4. **Make it pleasant for developers to integrate**
5. **Demonstrate the model with real sample apps**
6. **Only then widen ecosystem fit**

That means the project should evolve in layers, not as parallel chaos.

3. Phase Overview

Accord v1 should be delivered in seven phases:

1. **Foundation and Repo Setup**
2. **Protocol and Validation Core**
3. **Local Server and Persistence**
4. **Policy Engine and Approval Flow**
5. **Console and Inspectability**
6. **Developer SDK and Example Integrations**
7. **Polish, Hardening, and v0.1 Release Readiness**

Each phase should end with a concrete proof point.

4. Phase 0 — Project Foundation and Repo Setup

Goal

Create a clean monorepo, baseline tooling, and the minimal project shape needed for implementation.

Deliverables

- monorepo initialized
- package manager and workspace config chosen
- formatting/lint/test tooling established

- docs moved into repo structure
- initial README skeleton
- contributor setup instructions
- first ADRs created

Suggested Outputs

- `packages/` directory structure
- `examples/` directory scaffold
- `docs/` directory populated with current documents
- root scripts for build/test/dev

Proof of Completion

A contributor can clone the repo, install dependencies, and understand the workspace structure without reading implementation code.

Notes

This phase should stay light. Do not overinvest in CI or release automation yet.

5. Phase 1 — Protocol and Validation Core

Goal

Turn the protocol spec into executable types, schemas, and validators.

Deliverables

- TypeScript types for all v0.1 protocol objects
- schema validation for object creation
- shared enums and constants
- object relationship helpers
- basic serialization/deserialization utilities
- initial protocol tests

Core Objects

- `ScopeRequest`
- `GrantedScope`
- `ContextManifest`
- `ActionProposal`
- `ApprovalReceipt`
- `RevocationEvent`
- `AuditRecord`

Proof of Completion

Given valid JSON payloads, the protocol package accepts and validates them. Given invalid or incomplete payloads, validation fails clearly.

What This Phase Should Prove

The runtime grammar is real and consistent enough to build on.

Do Not Add Yet

- database
 - console UI
 - external adapters
 - broad SDK ergonomics
-

6. Phase 2 — Local Server and Persistence

Goal

Create the local control-plane runtime that stores protocol objects and exposes a minimal API.

Deliverables

- local server package
- SQLite storage layer
- migrations or schema bootstrap
- CRUD paths for core objects
- append-only audit recording behavior
- local API endpoints for scope, manifest, proposal, approval, revocation, and audit records

Suggested Internal Modules

- validation module
- storage layer
- scope service
- proposal service
- approval service
- audit service

Proof of Completion

A local client can create and persist a scope request, granted scope, manifest, proposal, receipt, and audit history through the server.

What This Phase Should Prove

Accord is not just a schema library. It has a working local state machine.

Do Not Add Yet

- complex policy language
 - polished UI
 - multiple storage backends
-

7. Phase 3 — Policy Engine and Approval Flow

Goal

Make the control plane meaningful by enforcing allow / require_approval / deny behavior.

Deliverables

- rules-based policy evaluator
- conservative default policies
- expiry and revocation enforcement
- proposal evaluation flow
- approval receipt issuance
- receipt verification rules
- failure paths for stale or denied decisions
- policy-driven audit events

Minimum Policy Behaviors

- expired scope denies
- revoked scope denies
- writes require approval by default
- low-risk read-like actions may allow automatically
- target-locked approval supported
- single-use receipt behavior supported where practical

Proof of Completion

A proposal classified as `require_approval` cannot execute until an approval receipt is issued. Denied, expired, revoked, or mismatched states fail closed.

What This Phase Should Prove

The project's core promise is actually enforced.

Key Tests

- missing approval blocks execution
 - expired approval blocks execution
 - revoked scope blocks execution
 - target mismatch blocks execution
 - policy deny blocks execution
-

8. Phase 4 — Console and Inspectability

Goal

Make the runtime legible to humans.

Deliverables

- minimal React console
- pending approval list
- proposal detail view
- context manifest detail view
- active scopes view
- revoke scope action
- audit history view
- clear state labels for pending / approved / denied / expired / revoked

UI Priorities

- readability over visual sophistication
- proposal-to-manifest linkage must be obvious
- scope expiry and revocation must be visible
- audit navigation must be easy enough for debugging

Proof of Completion

A user can inspect a pending proposal, see what context was used, approve or deny it, and later inspect the resulting history.

What This Phase Should Prove

Accord is understandable in practice, not just correct on paper.

Do Not Add Yet

- beautiful dashboarding
- analytics-heavy views
- consumer-grade onboarding flows

9. Phase 5 — Developer SDK and Example Integrations

Goal

Make Accord adoptable by external developers and prove the control flow in real app scenarios.

Deliverables

- first stable TypeScript SDK surface
- local config flow for connecting to server
- helpers for scope request, manifest publish, action proposal, awaiting decision, and receipt verification
- example integration #1: local coding agent
- example integration #2: research-to-publish agent
- integration docs and quickstart

Example Integration Requirements

Coding Agent Example

Should demonstrate: - scope request for project preferences - context manifest generation - file write or git action proposal - approval before side effect - audit trail visibility

Research Publisher Example

Should demonstrate: - scope request for writing preferences - proposal before publish/export/PR - approval flow - console visibility

Proof of Completion

A new developer can follow docs and add Accord to a sample local app in a few hours.

What This Phase Should Prove

Accord is a usable primitive, not just a reference implementation.

10. Phase 6 — Hardening, Documentation, and v0.1 Release Readiness

Goal

Stabilize the project enough for early public open-source release.

Deliverables

- regression tests for approval and scope failures
- fixture-based protocol tests
- clearer error messages
- README rewritten for external users
- contribution guide
- example walkthrough docs
- API reference draft
- release notes for v0.1
- issue templates and milestone labels

Safety-Driven Hardening Tasks

- verify receipt immediately before execution in examples
- ensure append-only audit assumptions are documented
- validate conservative defaults
- add warnings for integrators about bypass limitations

Proof of Completion

An external developer can understand what Accord is, run it locally, inspect the examples, and trust that the main control paths are tested.

What This Phase Should Prove

The project is ready for early adopters and thoughtful feedback.

11. Optional Phase 7 — Ecosystem Fit and Adapter Exploration

Goal

Explore how Accord plugs into adjacent ecosystems after the core model is solid.

Candidates

- MCP-oriented adapter
- HCP-alignment note or mapping doc
- richer policy config format
- additional example app

Caution

This phase should begin only after the control plane and examples already feel coherent.

Proof of Completion

Accord's relationship to adjacent standards becomes clearer without distorting the core project.

12. Milestone View

A simpler milestone view can help keep momentum.

Milestone A — Protocol Lives

Definition: Protocol objects, schemas, and validation exist and are tested.

Milestone B — Local State Machine Lives

Definition: Server + SQLite persistence + audit trail work locally.

Milestone C — Fail-Closed Approval Works

Definition: Policy + approval + verification reliably block unsafe execution.

Milestone D — Human Inspection Works

Definition: Console makes scope, manifest, proposal, receipt, and audit history visible.

Milestone E — Developer Integration Works

Definition: SDK + examples prove Accord can be adopted without unreasonable effort.

Milestone F — Early OSS Release Ready

Definition: Tests, docs, and examples are clean enough for outside use.

13. Suggested First 30-Day Focus

If we want a disciplined start, the first month should aim only for:

Weeks 1-2

- repo setup
- docs import
- protocol package
- validation package or internal module

- initial tests

Weeks 3–4

- server bootstrap
- SQLite schema
- scope/proposal/audit persistence
- minimal end-to-end local flow without full console polish

What Month 1 Should Achieve

By the end of the first month, the project should already prove a minimal vertical slice:

create scope → create manifest → propose action → require approval → issue receipt →
verify before execution → write audit history

That is far more important than shipping a nice UI early.

14. Suggested First 60-Day Focus

The second stretch should deepen usefulness.

Focus Areas

- stabilize policy logic
- build console views
- tighten status transitions
- add failure-mode tests
- define first SDK ergonomics
- start coding agent example

What Month 2 Should Achieve

By the end of this stretch, a developer should be able to watch the full approval flow in a browser console and understand the runtime model clearly.

15. Suggested First 90-Day Focus

The third stretch should aim for release-readiness, not feature explosion.

Focus Areas

- complete second example app
- tighten docs and quickstarts
- refine error messages

- harden audit and verification tests
- improve README and architectural clarity
- prepare v0.1 public release plan

What Month 3 Should Achieve

By the end of this stretch, Accord should be something you would be comfortable showing publicly as a serious open-source project.

16. Recommended ADR Topics

The roadmap implies a few immediate architecture decision records.

Suggested ADRs: - Why TypeScript-first - Why local-first for v1 - Why SQLite first - Why Accord is not a memory engine - Why approval is a first-class primitive - Why sample apps are proof, not product - Why conservative default policy matters

These ADRs will keep the repo intellectually clean as it grows.

17. What to Avoid During Execution

The roadmap will fail if we do the following too early:

- build a flashy landing page before the control flow works
- add five adapters before one example feels solid
- debate perfect protocol purity without implementation feedback
- over-engineer pluggability before the core data model stabilizes
- chase integrations that require sensitive user accounts too early
- broaden into “AI trust platform” language before the core use case proves itself

The discipline is to keep proving the same central idea from different angles:

AI can use context and still remain approval-aware, inspectable, and bounded.

18. Release Readiness Checklist for v0.1

Before calling anything a v0.1 release, the following should be true:

- protocol objects and validation are stable enough for early adoption
- local server is reliable for sample workflows
- policy engine enforces fail-closed behavior
- receipt verification is explicit and tested
- console shows proposal + manifest + scope + audit relationships clearly

- at least two example apps work end to end
- README explains purpose in under a minute
- docs explain integration, limits, and safety assumptions clearly

If these are not true, it is not time to call it a release.

19. Closing Statement

Accord's roadmap should not optimize for activity. It should optimize for proof.

Each phase should prove one increasingly important thing:

- the protocol is coherent,
- the runtime works,
- the approval boundary holds,
- the system is inspectable,
- developers can adopt it,
- and the project is honest about what it does and does not guarantee.

If we follow that sequence, Accord will grow like real infrastructure rather than a loose collection of ideas.